

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

Após um pré-processamento em tempo $\Theta(n)$, uma heap permite obter repetidamente o maior/menor elemento em tempo $\Theta(\log n)$. A inclusão, remoção ou alteração de elemento na heap é feita em tempo $\Theta(\log n)$.

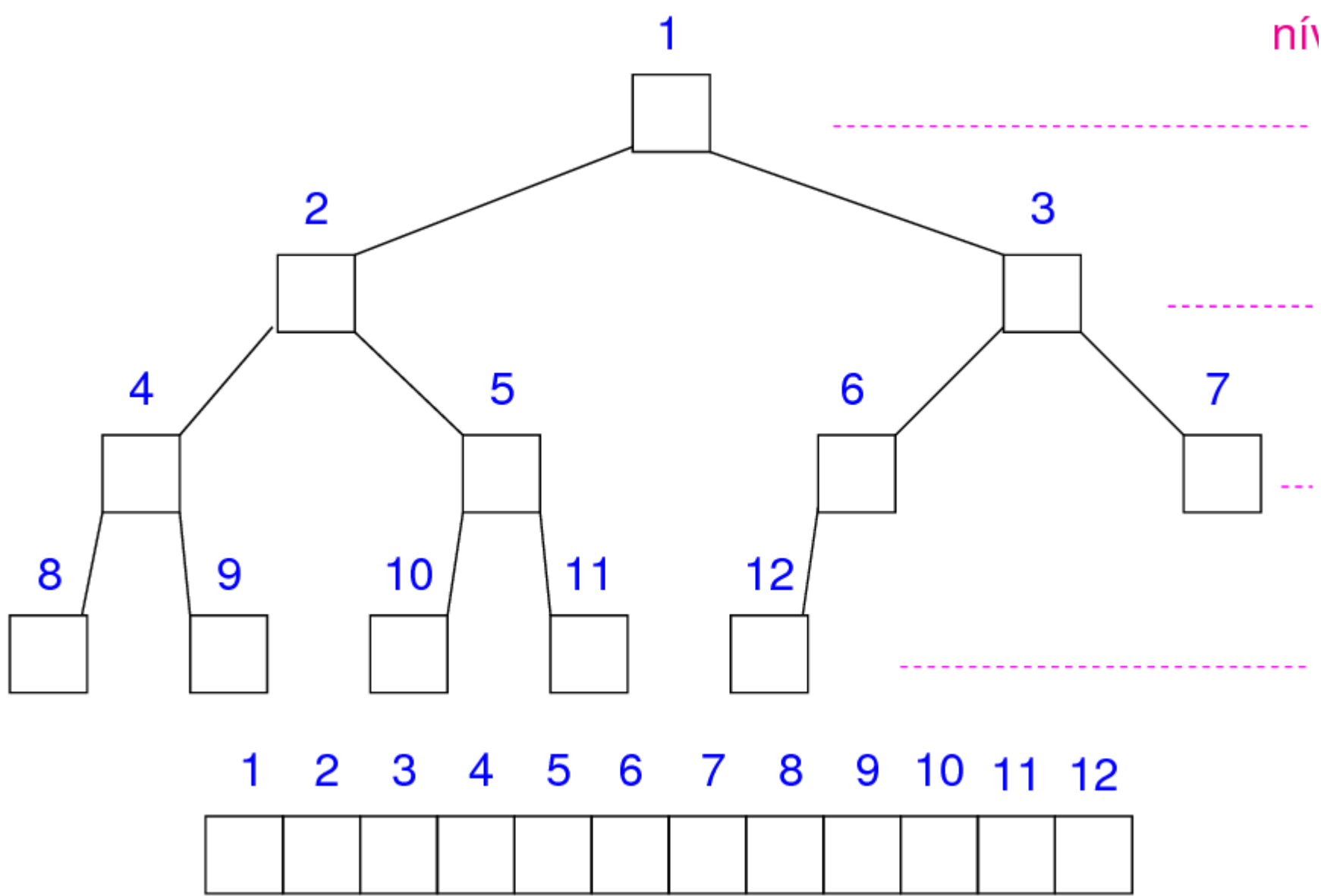
Usando vetor ordenado:

- Obter o maior/menor elemento seria em $\Theta(1)$.
- Porém, gastaríamos
 - $\Theta(n \log n)$ no pré-processamento (ordenar o vetor).
 - $\Theta(n)$ para inserir, remover ou alterar elemento.

Nos slides, este assunto está no arquivo [ordenacao-4up.pdf](#) da página 10 até a página 26. Os slides possuem a solução passo-a-passo dos exemplos.

Representação de heap em vetor

Uma heap é uma árvore binária completa (exceto talvez o último nível). Então, pode ser representada em vetor.



Pai de i está na posição $\lfloor i/2 \rfloor$. Nó 1 não tem pai.

Nó i tem $2i$ como filho esquerdo, e $2i+1$ como filho direito.

Nó i tem filho esquerdo se $2i \leq n$, e direito se $2i + 1 \leq n$.

Nó i é folha se não tem filhos, ou seja, $2i > n$.

Se i é ímpar, então o último nó com filho é $\lfloor n/2 \rfloor$, pois o dobro da posição central vale $n + 1$. Se i é par, então o último nó com filho é $n/2 = \lfloor n/2 \rfloor$.

Portanto, as folhas são $\lfloor n/2 \rfloor + 1, \dots, n - 1, n$.

Altura de um nó

Nós no nível p estão nas posições abaixo: (exceto talvez último nível)

$$2^p, 2^p + 1, \dots, 2^{p+1} - 1.$$

Logo,

$$2^p \leq i < 2^{p+1} \Rightarrow \log_2 2^p \leq \log_2 i < \log_2 2^{p+1} \Rightarrow p \leq \log_2 i < p + 1.$$

Portanto, o nó i pertence ao nível $p = \lfloor \log_2 i \rfloor$. Assim, como começamos no nível 0 o total de níveis vale $1 + \lfloor \log_2 n \rfloor$.

Altura do nó i : número de arestas de i até a folha mais distante. Folhas tem altura zero.
Como os níveis são preenchidos da esquerda pra a direita, podemos determinar o maior número de arestas até um seguindo sempre o filho da esquerda. Os filhos à esquerda partindo de i estão nas posições

$$2i, 2^2i, 2^3i, \dots, 2^hi,$$

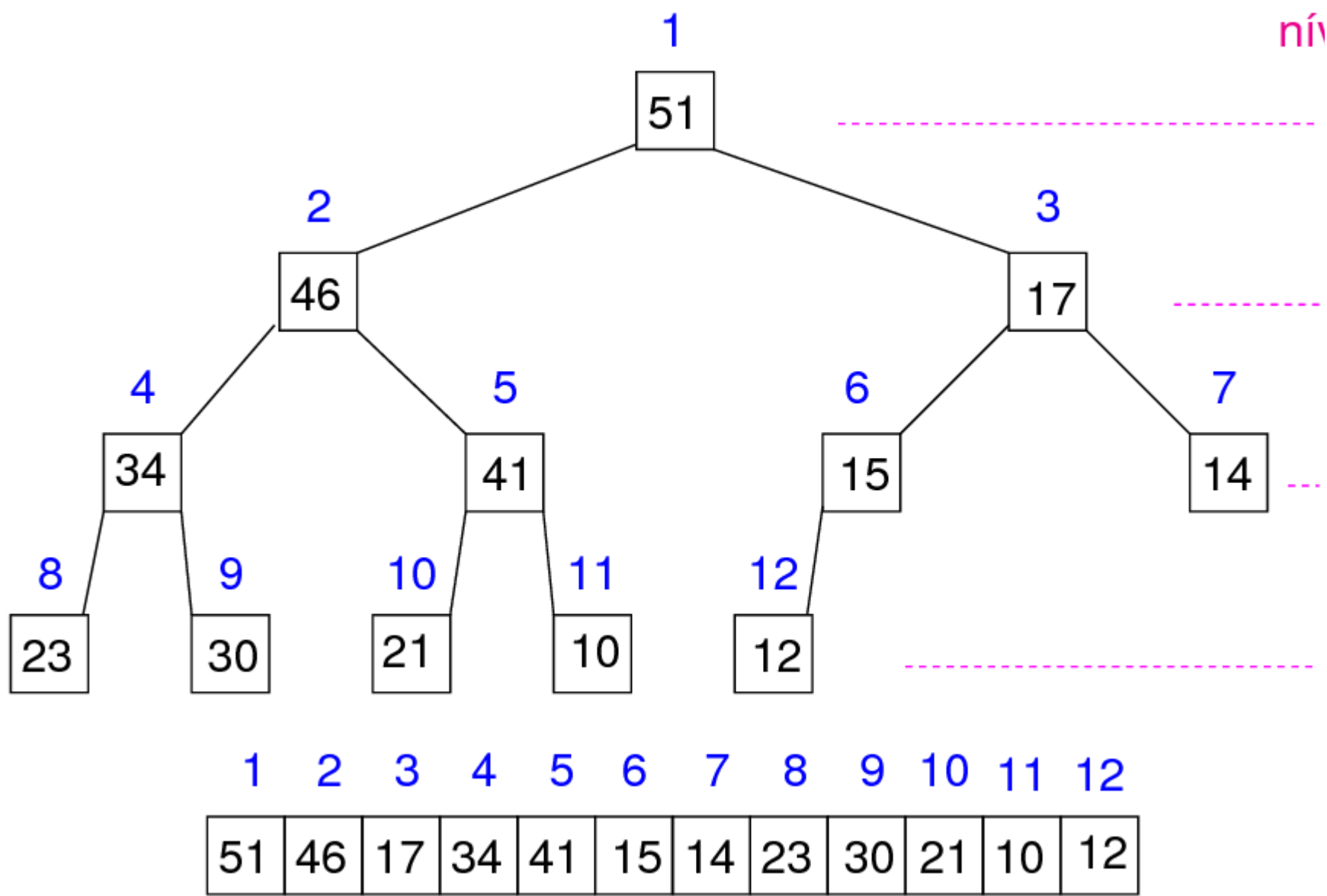
onde $2^hi \leq n < 2^{h+1}i$. Então,

$$\begin{aligned} \log_2(2^hi) \leq \log_2 n < \log_2(2^{h+1}i) &\Rightarrow h + \log_2 i \leq \log_2 n < (h + 1) + \log_2 i \\ &\Rightarrow h \leq \log_2(n/i) < h + 1 \end{aligned}$$

Portanto, a altura de i vale $h = \lfloor \log_2(n/i) \rfloor$.

Max-heaps

Um nó i satisfaz a **propriedade de max-heap** se o valor armazenado no pai é maior ou igual ao valor dos filhos, ou seja $A[i] \geq A[2i]$ e $A[i] \geq A[2i + 1]$. Uma árvore binária completa é **max-heap** se todo nó satisfaz a propriedade de max-heap. Portanto, por transitividade o maior elemento sempre está na raiz da max-heap. A definição de min-heap é similar: inverter a desigualdade.



Heapify

A operação **max-heapify** recebe uma árvore binária completa onde apenas o elemento da raiz não satisfaz a prop de max-heap, e transforma esta árvore em uma max-heap. É um algoritmo recursivo, onde a 1ª chamada deve ser **HEAPIFY(A, n, 1)** Note que a raiz está na posição 1.

MAX-HEAPIFY(A,n,i):	
esq <- 2*i; dir <- 2*i + 1; maior <- i	O(1)
se esq <= n e A[esq] > A[maior], então maior <- esq	O(1)
se dir <= n e A[dir] > A[maior], então maior <- dir	O(1)
se maior != i	O(1)
troque A[i] com A[maior]	O(1)
MAX-HEAPIFY(A,n,maior)	T(h-1)

$$T(h) = T(h - 1) + O(1) \in O(h).$$

Então, $T(h) \in O(\lg n)$, pois $h = \lfloor \log_2 n \rfloor$.

Exemplo: nos slides.

Make-heap

Como transformar um vetor inicial em uma heap (operação make-heap)?

Chame heapify do último até o 1o elemento. Assim, em cada chamada apenas a raiz não satisfaz a propriedade de heap. Como as folhas já satisfazem a propriedade, basta começar em $\lfloor n/2 \rfloor$.

MAKE-MAX-HEAP(A,n):	
para i <- piso(n/2) até 1 (ordem decrescente)	
MAX-HEAPIFY(A,n,i)	

$$T(n) \in O(n \lg n).$$

Exemplo: nos slides.

Na verdade é possível mostrar que $T(n) \in O(n)$.

Seja $S(h)$ a soma das alturas dos nós de uma árvore binária completa de altura h . O número de operações do hea aplicado no nó i é da ordem da altura de i . Como o make-heap chama o heapify para todos os nós com altura diferente de zero, temos que o make-heap realiza $O(S(h))$ operações. Portanto, basta mostrar que $S(h) \in O(n) = O(2^h)$, pois $h \in O(\lg n)$.

$S(h)$ pode ser obtida através da recorrência $S(0) = 0$ e $S(h) = 2S(h - 1) + h$, pois se tivermos a soma das alturas das duas subárvores filhas, basta acrescentar a altura da raiz (h). Vamos provar por indução que a solução desta recorrência vale $S(h) = 2^{h+1} - h - 2$.

Prova por indução:

Caso Base: $S(0) = 2^{0+1} - 0 - 2 = 0$ (v).

Hipótese: $S(h - 1) = 2^{h-1+1} - (h - 1) - 2 = 2^h - h - 1$.

$$S(h) = 2S(h - 1) + h = 2(2^h - h - 1) + h = 2^{h+1} - h - 2.$$

Heapsort

O heapsort utiliza uma **max-heap** para encontrar o maior elemento, inserindo no final do vetor. Em cada iteração ele troca o maior elemento com o 1o, reduz em 1 o tamanho da heap, e chama max-heapify na raiz.

HEAPSORT(A,n):	
MAKE-MAX-HEAP(A,n)	O(n)
para tam_heap <- n até 2 (decrescente)	O(n)
troca A[1] com A[tam_heap]	O(n)
MAX-HEAPIFY(A, tam_heap-1, 1)	O(n lg n)

$$T(n) \in O(n \lg n).$$

Exemplo: nos slides.

Fila com prioridades

Tipo abstrato de dados que consiste em uma coleção S de itens, cada um com uma prioridade, e as operações:

- MAXIMUM(S)**: retorna o elemento de S com maior prioridade.
- EXTRACT-MAX(S)**: remove e retorna o elemento de S com maior prioridade.
- INCREASE-KEY(S,x,p)**: aumenta a prioridade de x para p .
- DECREASE-KEY(S,x,p)**: reduz a prioridade de x para p .
- INSERT(S,x,p)**: insere em S elemento x com prioridade p .

Implementação com heap

```
MAXIMUM(A,n): // O(1)
    retorne A[1]

EXTRACT-MAX(A,n): // O(lg n)
    max <- A[1]
    copie A[n] para A[1]
    MAX-HEAPIFY(A, n-1, 1)
    decmente n
    retorne max

INCREASE-KEY(A,i,p): // O(lg n). Assume que p >= A[i].
    A[i] <- p
    enquanto i > 1 e A[i] > A[piso(i/2)] // enquanto maior que o pai
        troca A[i] com A[piso(i/2)]
        i <- piso(i/2)

DECREASE-KEY(A,n,i,p): // O(lg n). Assume que p <= A[i].
    A[i] <- p
    MAX-HEAPIFY(A,n,i)

INSERT(A,n,p): // O(lg n). Assume que A tem espaço p/ novo elemento.
    INCREASE-KEY(A, n+1, p)
    incremente n
```

Ex.: Código de Huffman.

◀ [Vídeo: fila de prioridade \(7min\)](#)

Seguir para...

[Vídeo: tabelas hash \(12min\)](#) ▶

©2020 – Universidade Federal do Ceará – Campus Quixadá.
Todos os direitos reservados.
Av. José de Freitas Queiroz, 5003
Cedro – Quixadá – Ceará CEP: 63902-580
Secretaria do Campus: (88) 3411-9422

 [Baixar o aplicativo móvel.](#)